

Face Recognition Attendance System

A Computer Vision Project built with OpenCV (Haar Cascade + LBPH)

Project Report

Domain	Computer Vision
Project Type	Build Your Own Project (Flipped Course Evaluation)
Interface	Command Line Interface (CLI) — no GUI required
Core Libraries	OpenCV (opencv-contrib-python), NumPy, pandas, matplotlib, SQLite
Repository	See README.md for the GitHub repository link
Report Generated	September 2026

1. Abstract

This project implements a complete, offline, CPU-only **Face Recognition Attendance System** using classical computer-vision techniques in OpenCV. The system detects human faces with a Haar Cascade classifier, recognizes enrolled individuals with an LBPH (Local Binary Patterns Histograms) model, and automatically logs attendance to a SQLite database, with duplicate-prevention (one attendance mark per person per day) enforced at the schema level. The entire workflow — enrollment, training, recognition against a webcam / video file / image, and analytics reporting (CSV + charts) — is exposed through a single, well-documented command-line interface, satisfying a strict 'fully executable via terminal, no GUI required' constraint. The project also ships an automated pytest suite (21 tests), structured logging, a custom exception hierarchy for clean error handling, and a privacy-conscious synthetic-data generator so the full pipeline can be demonstrated without requiring a webcam or real biometric photographs in the repository.

2. Problem Statement & Objectives

Manual, roll-call-based attendance is slow, interruptive, and vulnerable to proxy attendance (one person answering for another). A camera-based system that automatically detects and recognizes enrolled individuals removes both problems: attendance can only be marked when a registered face is actually detected and confidently recognized by the system, and the process takes seconds rather than minutes of roll-call.

2.1 Objectives

- Detect human faces in real time from a webcam, a recorded video, or a still image.
- Enroll new individuals either via live webcam capture or from existing photographs.
- Train a lightweight, CPU-only face-recognition model on the enrolled gallery.
- Recognize individuals and automatically log attendance, preventing duplicate same-day entries.
- Generate daily and date-range attendance reports with CSV export and visual charts.
- Deliver the whole system as a single, dependency-light, fully CLI-executable Python application with automated tests, structured logging, and clear error handling.

2.2 Scope & Design Trade-offs

The project deliberately uses **Haar Cascade detection** and **LBPH recognition** instead of a deep-learning face-embedding pipeline (e.g. FaceNet/ArcFace with a DNN detector). Both are bundled with / trivially installable alongside OpenCV, require no external model downloads, run entirely on CPU, and train in well under a second for a small enrolled gallery — all important properties for a project that must be reproducible in any evaluator's terminal without GPU access or large downloads. The trade-off is lower recognition accuracy than a modern deep embedding model at large gallery sizes; this is documented as a known limitation and a natural direction for future work (Section 8).

3. Technology Stack

Component	Technology	Reason for choice
Language	Python 3.10+	Fast to develop in; OpenCV's Python bindings are first-class
Face detection	OpenCV Haar Cascade (haarcascade_frontalface_default.xml)	Ships with OpenCV; no external download; fast on CPU
Face recognition	OpenCV-contrib LBPH (cv2.face.LBPHFaceRecognizer)	Lightweight, interpretable, CPU-only, well suited to small galleries
Persistence	SQLite (stdlib sqlite3)	Zero-config relational storage with constraints for duplicate-prevention
Reporting	pandas / csv + matplotlib	Structured export and visual analytics
Configuration	PyYAML	Human-readable, single source of truth for tunables
Testing	pytest	Industry-standard, fixture-based isolated testing
CLI	argparse (stdlib)	No extra dependency; sub-command support out of the box

4. System Architecture

The codebase is organized into single-responsibility layers: a CLI layer for user interaction, an orchestration layer that coordinates the computer-vision components, the CV components themselves, a persistence layer, and a reporting layer, with configuration and logging injected as cross-cutting concerns.

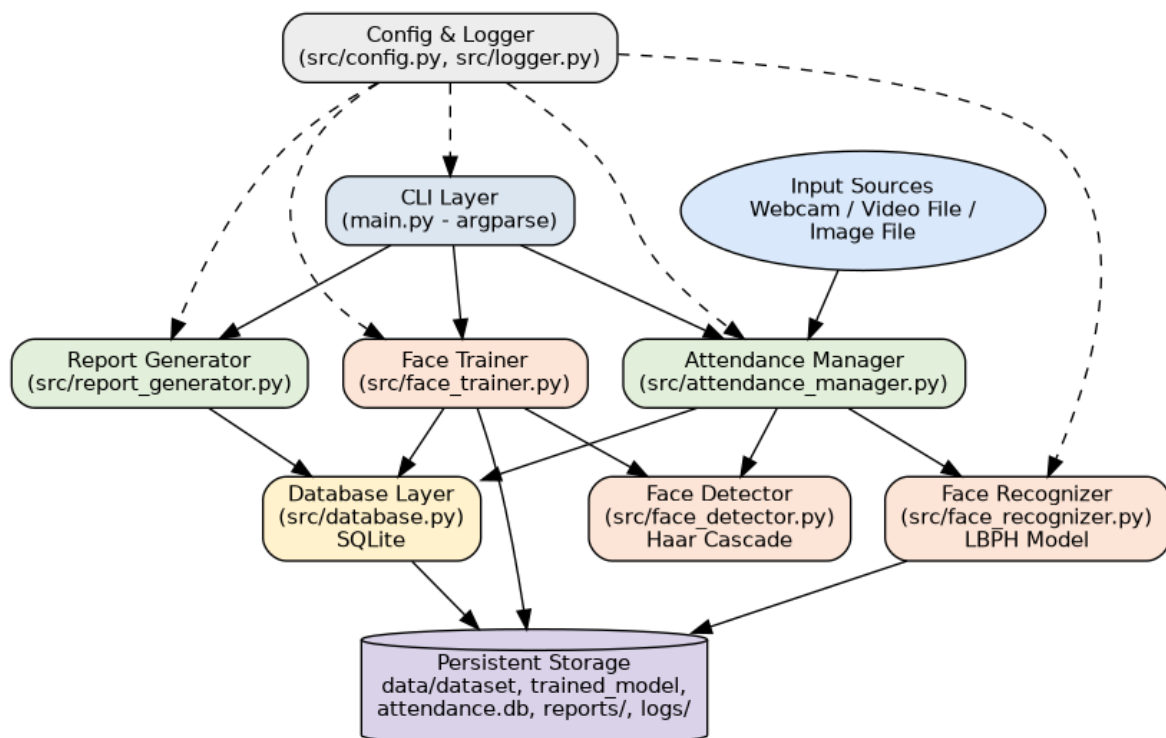


Figure 1: System architecture — module responsibilities and dependencies.

4.1 Module Responsibilities

Module	Responsibility
main.py	CLI entry point: argument parsing and command dispatch

Module	Responsibility
src/config.py	Typed configuration loader; creates required directories
src/logger.py	Console + rotating file logging setup
src/exceptions.py	Custom exception hierarchy for clean error handling
src/database.py	SQLite schema, CRUD, attendance de-duplication
src/face_detector.py	Haar-cascade face localization and normalization
src/face_trainer.py	Dataset building (webcam/photos) + LBPH training
src/face_recognizer.py	LBPH inference — identity + confidence per face
src/attendance_manager.py	Orchestrates detect→recognize→mark-attendance over webcam/video/image
src/report_generator.py	CSV export + matplotlib attendance charts

4.2 Workflow / Process Flow



Figure 2: End-to-end process flow from enrollment through reporting.

4.3 Use Case Diagram

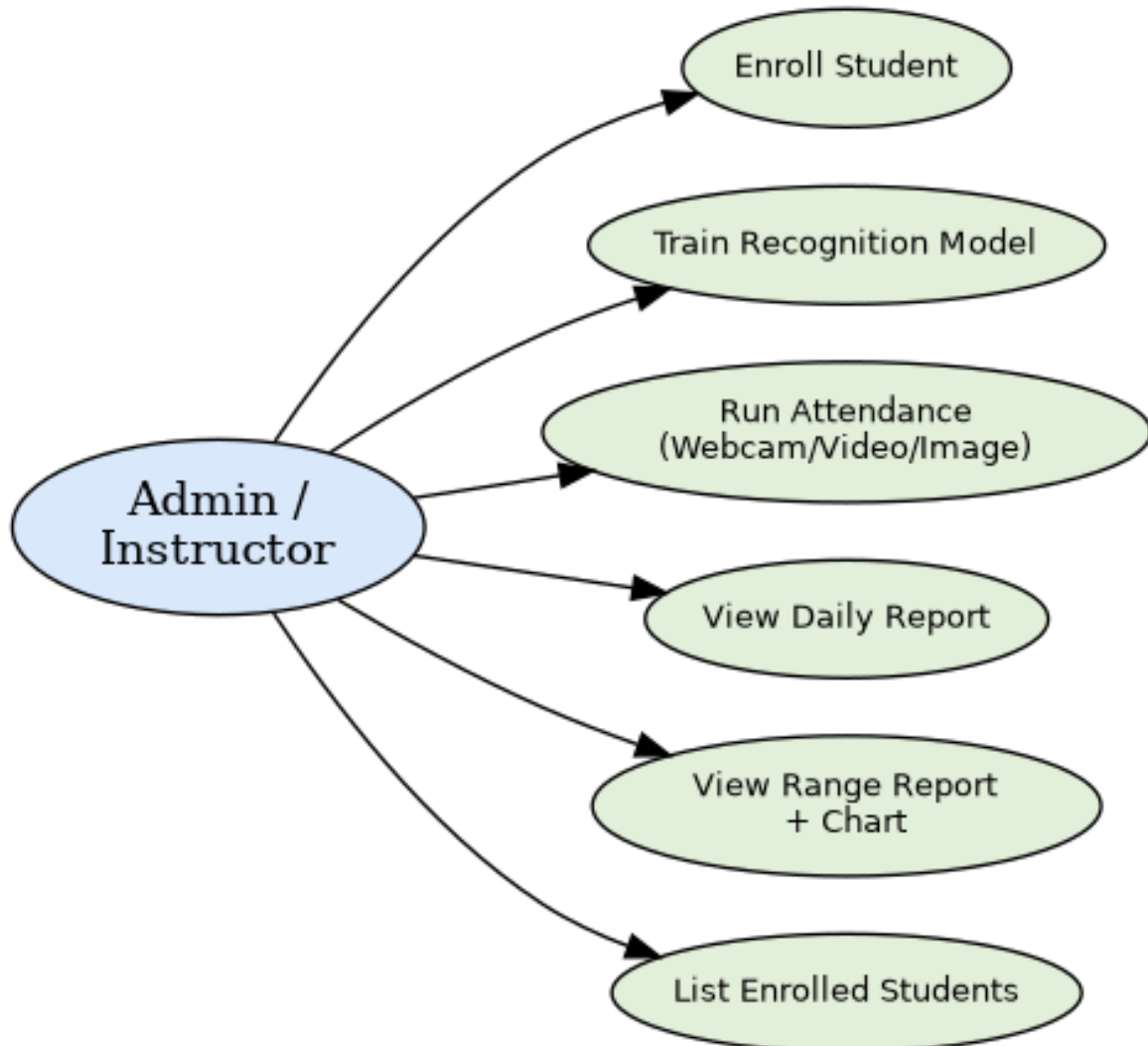


Figure 3: Use cases available to the Admin/Instructor actor via the CLI.

4.4 Class Diagram

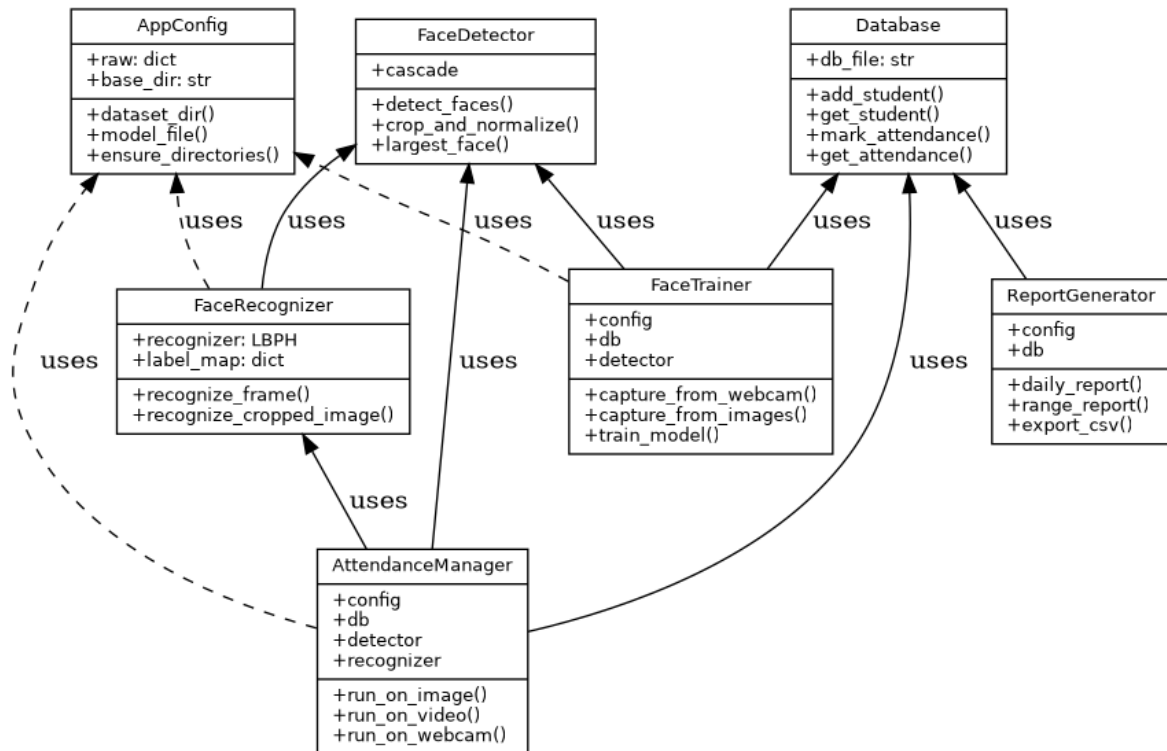


Figure 4: Simplified class diagram showing module classes and their dependencies.

4.5 Sequence Diagram — Recognize & Mark Attendance

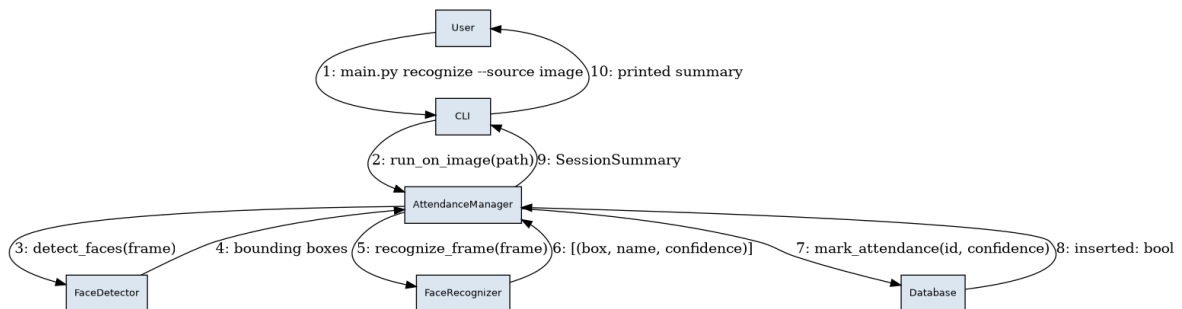


Figure 5: Sequence of calls for `python main.py recognize --source image ...`.

4.6 Database Design

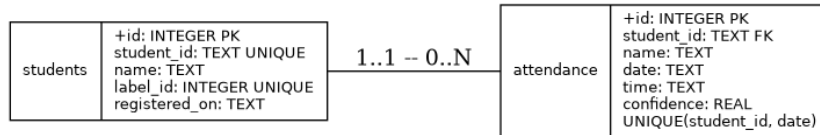


Figure 6: Entity-Relationship diagram — students and attendance tables.

The `UNIQUE(student_id, date)` constraint on the attendance table is what makes `mark_attendance()` idempotent: a second recognition of the same person on the same day fails the insert with an `IntegrityError`, which the database layer catches and reports as an ordinary 'already marked' outcome rather than a duplicate row or an unhandled crash.

5. Implementation Details

5.1 Face Detection

Faces are located using OpenCV's bundled Haar Cascade classifier (`haarcascade_frontalface_default.xml`), run on a histogram-equalized grayscale copy of each frame for lighting robustness. Detected regions are cropped, re-equalized, and resized to a fixed size (200x200 px by default) so every sample fed into training/recognition is geometrically consistent.

5.2 Enrollment & Training

Two enrollment paths are supported so the system is usable without a webcam: live webcam capture (samples the largest detected face across successive frames) and folder-of-photos enrollment, with an additional `--cropped` mode for datasets that are already face-only images (common in public benchmark datasets). Training collects every saved sample across all enrolled students, fits an LBPH recognizer (`cv2.face.LBPHFaceRecognizer_create()`), and persists both the trained model and a JSON label→identity map to disk.

5.3 Recognition & Attendance Logging

For each detected face, the LBPH model returns a predicted label and a confidence value (an LBPH distance — lower means a closer match). Predictions above the configured `confidence_threshold` are reported as Unknown instead of being forced onto the nearest label, trading a small amount of recall for a much lower false-accept rate — an appropriate trade-off for an attendance-marking system. Every confidently recognized face triggers an attempted attendance insert, which the database layer silently no-ops if that student was already marked present earlier the same day.

5.4 Reporting & Analytics

The reporting module queries the SQLite attendance table for a single day or an arbitrary date range, exports the result as CSV, and — for range reports — renders a matplotlib bar chart of attendance-day counts per student using the non-interactive 'Agg' backend so it works in headless/CLI-only environments.

5.5 Error Handling & Logging

A dedicated exception hierarchy (`src/exceptions.py`) — `CameraError`, `NoFaceDetectedError`, `ModelNotTrainedError`, `StudentAlreadyExistsError`, and others — is caught at the CLI boundary in `main.py` and converted into clear, actionable console messages instead of raw stack traces. Every significant action (enrollment, training, recognition, attendance marks) is logged with a timestamp to both the console and a rotating log file (`logs/app.log`) for later auditing/debugging.

5.6 Privacy-Conscious Demo Data

Real face photographs are personal biometric data and are intentionally never committed to this repository. `scripts/generate_synthetic_faces.py` procedurally draws simple, privacy-safe placeholder face images (with per-identity variation in skin tone, eye spacing, hairstyle, and slight rotation/noise) purely so the full pipeline can be demonstrated end-to-end without a webcam or real photos. Real-world usage enrolls actual people via `--source webcam` or the user's own photographs.

6. Testing

An automated pytest suite (21 tests across 5 files) validates the system's core behaviour in isolation from any real user data, using pytest fixtures that build a temporary, disposable copy of the app's configuration and directory layout for every test.

- **test_database.py** — student registration, duplicate-enrollment rejection, label-ID assignment, once-per-day attendance de-duplication, date-range queries.
- **test_config.py** — configuration loading and automatic directory bootstrapping.
- **test_face_detector.py** — detection on a blank frame returns no boxes; face cropping/normalization produces the configured output size; largest-face selection logic.
- **test_pipeline_integration.py** — the full enroll → train → recognize pipeline against the synthetic dataset generator (with the detector's bounding-box step deterministically stubbed, since the Haar cascade is trained on real photographic faces and will not reliably fire on procedurally-drawn placeholder images — a documented, deliberate testing methodology, not a defect).
- **test_report_generator.py** — CSV export correctness and chart generation for both daily and range reports.

Automated Test Suite

```
$ python -m pytest tests/ -v
```

```
===== test session starts =====
```

```
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
```

```
cachedir: .pytest_cache
```

```
rootdir: /home/claude/face-attendance-cv
```

```
collecting ... collected 21 items
```

```
tests/test_config.py::test_load_config_creates_directories PASSED [ 4%]
tests/test_config.py::test_face_size_and_min_size_are_tuples PASSED [ 9%]
tests/test_database.py::test_add_and_get_student PASSED [ 14%]
tests/test_database.py::test_duplicate_enrollment_raises PASSED [ 19%]
tests/test_database.py::test_label_ids_increment PASSED [ 23%]
tests/test_database.py::test_list_students_sorted_by_name PASSED [ 28%]
tests/test_database.py::test_mark_attendance_success PASSED [ 33%]
tests/test_database.py::test_mark_attendance_once_per_day PASSED [ 38%]
tests/test_database.py::test_mark_attendance_unknown_student_raises PASSED [ 42%]
tests/test_database.py::test_get_attendance_date_filter PASSED [ 47%]
tests/test_face_detector.py::test_detect_faces_on_blank_frame_returns_empty PASSED [ 52%]
tests/test_face_detector.py::test_crop_and_normalize_output_shape PASSED [ 57%]
tests/test_face_detector.py::test_largest_face_picks_biggest_box PASSED [ 61%]
tests/test_face_detector.py::test_largest_face_empty_list PASSED [ 66%]
tests/test_pipeline_integration.py::test_enroll_train_recognize_pipeline PASSED [ 71%]
tests/test_pipeline_integration.py::test_training_fails_with_no_students PASSED [ 76%]
tests/test_pipeline_integration.py::test_capture_from_images_missing_folder_raises PASSED [ 80%]
tests/test_report_generator.py::test_daily_report_empty PASSED [ 85%]
tests/test_report_generator.py::test_daily_report_with_records PASSED [ 90%]
tests/test_report_generator.py::test_range_report_generates_chart PASSED [ 95%]
tests/test_report_generator.py::test_last_n_days_range_length PASSED [100%]
```

```
===== 21 passed in 0.76s =====
```

Figure 7: Full test suite passing (21/21).

7. Results & Demonstration

The following screenshots are captured from actual runs of this codebase against the bundled synthetic demo dataset (not mockups).

```
Enrollment + Model Training

$ python main.py enroll --id STU001 --name "Aditi Sharma" --source images --path data/sample_faces/person_a --cro
[OK] Enrolled 'Aditi Sharma' (STU001) with 12 face sample(s).

    Run `python main.py train` next to (re)build the recognition model.

$ python main.py enroll --id STU002 --name "Rohan Verma" --source images --path data/sample_faces/person_b --cro
[OK] Enrolled 'Rohan Verma' (STU002) with 12 face sample(s).

    Run `python main.py train` next to (re)build the recognition model.

$ python main.py train

[OK] Model trained on 2 student(s). Saved to /home/claude/face-attendance-cv/data/trained_model/lbph_model.yml
```

Figure 8: Enrolling two identities and training the LBPH model.

```
Recognition + Attendance Marking

$ python main.py recognize --source image --path data/sample_faces/person_a/010.png --cropped
[OK] Processed 1 frame(s), saw 1 face detection(s).

    Attendance marked for: STU001

$ python main.py recognize --source image --path data/sample_faces/person_b/011.png --cropped
[OK] Processed 1 frame(s), saw 1 face detection(s).

    Attendance marked for: STU002

$ python main.py list-students

Student ID      Name                Registered On
-----
STU001          Aditi Sharma        2026-09-16T06:15:53
STU002          Rohan Verma         2026-09-16T06:15:54
```

Figure 9: Recognition marking attendance, and listing enrolled students.

Attendance Report (Range)

```
$ python main.py report --type range --days 7
```

```
[OK] Range report 2026-09-10 -> 2026-09-16: 2 record(s)
```

- Aditi Sharma: 1 day(s) present
- Rohan Verma: 1 day(s) present

CSV saved to /home/claude/face-attendance-cv/reports/attendance_2026-09-10_to_2026-09-16.csv

Chart saved to /home/claude/face-attendance-cv/reports/attendance_chart_2026-09-10_to_2026-09-16.png

Figure 10: A 7-day range attendance report.

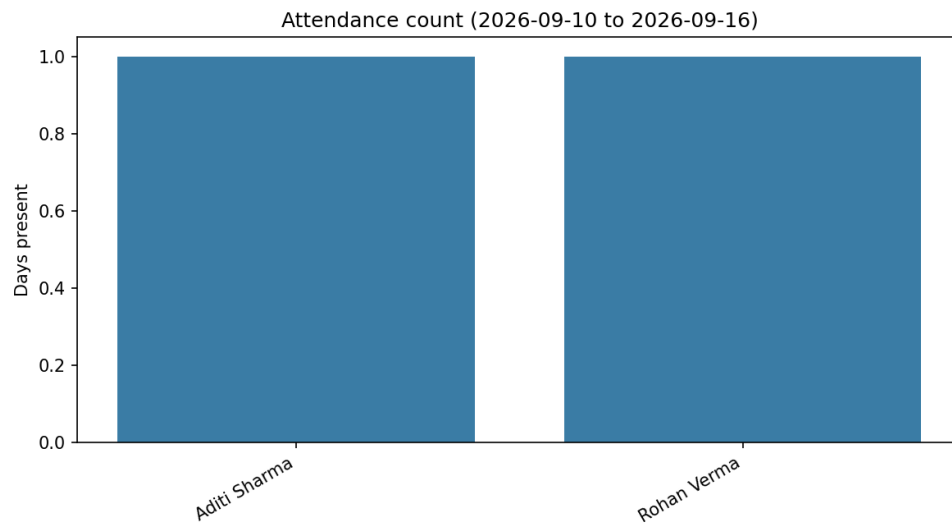


Figure 11: Auto-generated attendance bar chart (matplotlib).

8. Non-Functional Requirements Addressed

Requirement	How it is addressed
Performance	CPU-only Haar Cascade + LBPH; configurable frame-skip for video processing
Reliability / Error handling	Custom exception hierarchy caught at the CLI boundary; no unhandled crashes on bad input, missing files, etc.
Security / Privacy	No real biometric data committed to the repo; enrollment data stays local and is gitignored
Usability	Single self-documenting CLI (<code>-h</code> on every command); zero required configuration to get started
Maintainability	Small single-responsibility modules, docstrings explaining design rationale, externalized config.yaml
Logging & Monitoring	Every action logged with timestamps to console + rotating log file
Scalability	Adding a student is $O(1)$; LBPH retrain the full gallery in well under a second at this scale
Testability	21 automated pytest tests with isolated fixtures, runnable with one command

9. Conclusion & Future Work

This project delivers a complete, self-contained, CLI-driven face recognition attendance system covering the full computer-vision pipeline from detection through recognition, persistence, and analytics reporting. It is fully reproducible offline on CPU-only hardware, has no GUI dependency, and is backed by an automated test suite, structured logging, and clear error handling — meeting the functional and non-functional requirements set out in the problem statement (Section 2).

Future Enhancements

- Replace LBPH with a deep-learning face-embedding model (e.g. FaceNet/ArcFace) for higher recognition accuracy at larger gallery sizes.
- Add liveness/anti-spoofing checks (blink detection, texture analysis) to prevent a held-up photograph from marking attendance.
- A lightweight read-only web dashboard for browsing historical reports (the CLI + CSV/PNG output was chosen deliberately to satisfy the terminal-executability requirement).
- Multi-camera / multi-room deployment sharing a single central database.

10. References

- OpenCV Documentation — Cascade Classifier: https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
- OpenCV Documentation — Face Recognition (LBPH): https://docs.opencv.org/4.x/da/d60/tutorial_face_main.html
- Ahonen, T., Hadid, A., & Pietikäinen, M. (2006). Face Description with Local Binary Patterns: Application to Face Recognition. IEEE TPAMI.

- Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. CVPR.
- Python sqlite3 Standard Library Documentation: <https://docs.python.org/3/library/sqlite3.html>